

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

**ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2 "Взаимодействие
с внешним API и эмуляция серверной части"**

Выполнил:
Демьянов Фёдор

Группа: J3214

Проверил:
Добряков Д. И.

Санкт-Петербург

2023 г.

Введение

Цель работы - проектирование и разработка клиент-серверной архитектуры веб-приложения с организацией локального REST API на базе `json-server`, реализация асинхронного сетевого взаимодействия средствами нативного Fetch API с использованием синтаксиса `async/await`, построение подсистемы клиентской аутентификации и сессионного контроля, а также интеграция внешнего публичного сервиса NASA APOD.

В качестве предметной области продолжено развитие сервиса частных космических полетов «NovaTransit», спроектированного в рамках лабораторной работы № 1. Если на первом этапе фокус был сосредоточен на адаптивной верстке, компонентной базе Bootstrap 5 и статической структуре документов, то во второй лабораторной работе проект трансформируется в динамическое клиент-серверное веб-приложение. Специфика космических экспедиций (учет свободных мест, персонализация досье кадетов, бронирование полетных манифестов, передача телеметрии жилых кают) требует полноценного моделирования работы с базой данных и обеспечения надежного обмена данными по протоколу HTTP.

Для достижения поставленной цели были сформулированы и решены следующие задачи:

1. Спроектировать реляционную структуру данных в формате `db.json`, формализующую сущности пользователей (`users`), экспедиций (`tours`) и полетных манифестов (`bookings`).
2. Развернуть и сконфигурировать локальный сервер `json-server`, настроив параметры запуска и правила игнорирования файлов в рабочей среде.
3. Разработать модульный HTTP-клиент (`js/api.js`), инкапсулирующий базовую конфигурацию сетевых запросов, сериализацию JSON, подстановку служебных заголовков и сквозную обработку ошибок.
4. Создать модуль аутентификации и сессионного контроля (`js/auth.js`), включающий регистрацию новых исследователей, вход со сверкой учетных записей, барьер авторизации для защищенных экранов и динамическую адаптацию навигационной панели.
5. Перевести реестр экспедиций главной страницы (`js/catalog.js`) на асинхронную загрузку с сервера с сохранением адаптивной многопараметрической фильтрации.
6. Реализовать динамическую инициализацию страницы конкретного маршрута (`js/tour.js`) и отправку сформированных контрактов на сервер методом `POST`.
7. Организовать личный кабинет исследователя (`js/dashboard.js`) с загрузкой персонального манифеста рейсов и частичным обновлением систем жизнеобеспечения каюты методом `PATCH` без перезагрузки документа.
8. Интегрировать модуль взаимодействия со сторонним сервисом NASA APOD (`js/nasa.js`), настроив каскадный механизм обхода сетевых ограничений и политик CORS.

1. Проектирование схемы данных и организация Mock API

В рамках первой части работы я перевел сервис NovaTransit со статических данных на полноценное клиент-серверное взаимодействие. Для эмуляции серверной части и базы данных я выбрал инструмент `json-server`. Он позволяет за считанные минуты развернуть полноценный REST API на основе обычного JSON-файла, поддерживая стандартные HTTP-методы, фильтрацию, сортировку и автоматическую генерацию идентификаторов.

1.1. Реляционная модель данных `db.json`

Для хранения информации я спроектировал файл `db.json`, разделив предметную область на три взаимосвязанные коллекции: пользователей, экспедиции и оформленные бронирования.

- **Коллекция `users` (Пользователи и досье кадетов)**
Хранит учетные записи исследователей. Помимо базовых полей авторизации (логин, почтовый адрес, пароль), модель включает биометрический код кадета (`code` формата `NT-XXXX-X`), статус допуска (`class`), флаг участия в закрытом клубе (`isDeepSpaceMember`) и ссылку на сгенерированный аватар. Сюда же вынесены объекты `cabinSettings` (состояние тумблеров гравитации, рациона питания и затемнения иллюминаторов) и `trainingProgress` (прогресс предполетных нормативов). Это позволяет хранить индивидуальные параметры каюты каждого пользователя прямо в его профиле на сервере.
- **Коллекция `tours` (Реестр экспедиций)**
Содержит полную спецификацию космических маршрутов, которая в первой лабораторной работе была зашита в разметку каталога и JS-объект страницы тура. Запись включает технический шифр полета (`code`), ценовую категорию, текстовые анонсы, набор фильтров (направление, длительность, бюджет), технические параметры перегрузок и радиации в объекте `specs`, а также массивы слайдов фотохроники (`slides`) и контрольных точек полета (`timeline`).
- **Коллекция `bookings` (Полетные манифесты)**
Выступает связующим звеном между пользователем и выбранным маршрутом. Запись создается при оформлении заявки и содержит внешний ключ `userId`, связывающий бронь с конкретным кадетом. Помимо идентификаторов, манифест фиксирует дату старта, назначенный посадочный шлюз, категорию каюты, итоговую стоимость с учетом дополнительных опций и уникальный серийный номер электронного сертификата (`serialNo`).

Ниже приведен фрагмент структуры базы данных `db.json`, демонстрирующий организацию основных сущностей:

```
JSON
{
  "users": [
    {
      "id": 1,
      "name": "John Doe",
      "email": "commander@novatransit.com",
```

```
    "password": "password123",
    "code": "NT-8492-X",
    "class": "Silver Class",
    "isDeepSpaceMember": true,
    "avatarUrl":
"https://ui-avatars.com/api/?name=John+Doe&background=1a1e29&color=c8d0d
b&size=96",
    "cabinSettings": {
      "gravity": true,
      "hypoallergenicFood": false,
      "windowDimming": true
    }
  }
],
"tours": [
  {
    "id": "mars",
    "code": "EXP-04 // MARS",
    "badge": "VIP Explorer",
    "badgeClass": "badge-gold",
    "title": "Марсианский транзит",
    "price": 8900000,
    "destination": "mars",
    "duration": "long",
    "durationText": "180 дней",
    "budget": "vip",
    "specs": {
      "distance": "78 млн км",
      "gravity": "0.38 G",
      "gforce": "До 4.5 G",
      "radiation": "1.20 Зв"
    },
    "dates": ["02 апреля 2027", "18 сентября 2027"]
  },
],
"bookings": [
  {
    "id": 1,
    "userId": 1,
    "tourId": "mars",
    "tourCode": "EXP-04",
    "route": "Земля → Марс",
    "origin": "EAR",
    "destination": "MARS",
    "date": "12.04.2025",
    "ship": "Nova Cruiser II",
```

```

    "status": "Завершен",
    "statusBadge": "badge-silver"
  }
]
}

```

1.2. Спецификация REST API

Взаимодействие клиентской части с мок-сервером построено по архитектурному стилю REST. Сервер обрабатывает стандартные операции чтения, создания и модификации ресурсов, возвращая ответы в формате JSON.

В таблице 1 представлена сводная спецификация эндпоинтов локального сервера, задействованных в приложении.

Таблица 1 - Спецификация эндпоинтов локального Mock API ([json-server](#))

Метод HTTP	Эндпоинт	Назначение операции	Тело запроса (Payload)	Код ответа
GET	/users	Поиск и аутентификация исследователя по email и паролю	Отсутствует	200 OK
GET	/users/:id	Запрос актуального профиля кадета по идентификатору	Отсутствует	200 OK
POST	/users	Регистрация нового кадета с первичными настройками каюты	JSON с данными нового пользователя	201 Created
PATCH	/users/:id	Частичное обновление	{"cabinSettings": {...}}	200 OK

		параметров каюты (<code>cabinSettings</code>)		
GET	<code>/tours</code>	Получение полного реестра космических экспедиций	Отсутствует	200 OK
GET	<code>/tours/:id</code>	Запрос детальной информации по конкретному направлению	Отсутствует	200 OK / 404 Not Found
GET	<code>/bookings</code>	Получение манифестов с фильтрацией по кадету (<code>?userId=:id</code>)	Отсутствует	200 OK
POST	<code>/bookings</code>	Создание новой записи бронирования места в экспедиции	JSON с параметрами рейса и кадета	201 Created

1.3. Конфигурация рабочей среды

Для изоляции зависимостей второй лабораторной работы я инициализировал отдельное Node.js-окружение внутри директории `lab_02`.

В качестве сервера я установил пакет `json-server` фиксированной версии `0.17.4`. Выбор именно этой версии обусловлен ее стабильностью: в версиях линейки 1.0 разработчики библиотеки кардинально изменили синтаксис роутинга и фильтрации данных, что часто приводит к несовместимости стандартных REST-запросов.

В файле `package.json` я прописал скрипт запуска сервера с ключами отслеживания изменений файла на диске в реальном времени (`--watch`) и фиксацией порта (`--port 3000`):

```
JSON
{
  "name": "lab_02",
  "version": "1.0.0",
  "scripts": {
```

```
"server": "json-server --watch db.json --port 3000"
},
"dependencies": {
  "json-server": "^0.17.4"
}
}
```

При практической отладке возникла техническая проблема: расширение Live Server в VS Code по умолчанию отслеживало все файлы в рабочем пространстве. Каждый раз, когда клиент отправлял запрос **PATCH** или **POST**, **json-server** перезаписывал файл **db.json**. Live Server мгновенно фиксировал изменение файла на диске и принудительно перезагружал всю HTML-страницу в браузере. В результате страница мерцала, сессия на доли секунды сбрасывалась до шаблонных данных, а всплывающие уведомления Bootstrap Toast исчезали еще до завершения анимации появления.

Для решения этой проблемы я создал файл конфигурации **.vscode/settings.json** в корне рабочей области репозитория и добавил глобальное правило исключения:

```
JSON
{
  "liveServer.settings.ignoreFiles": [
    "**/db.json"
  ]
}
```

Благодаря маске ****/db.json** Live Server перестал реагировать на модификацию базы данных на любой глубине каталогов. Это позволило локальному серверу бесшовно сохранять данные в фоне без единой нежелательной перезагрузки интерфейса.

```
> lab2@1.0.0 server
> json-server --watch db.json --port 3000
```

```
\{^_^}/ hi!
```

```
Loading db.json
Done
```

Resources

```
http://localhost:3000/users
http://localhost:3000/tours
http://localhost:3000/bookings
```

Home

```
http://localhost:3000
```

Рисунок 1 - Запущенный экземпляр **json-server** в терминале с перечнем доступных **REST-ресурсов**

2. Клиент-серверная архитектура и подсистема авторизации

Чтобы не смешивать верстку, манипуляции с DOM-деревом и низкоуровневые HTTP-запросы, я разделил логику на независимые модули. Основой сетевого слоя стали два взаимосвязанных скрипта: `js/api.js`, отвечающий за отправку асинхронных запросов, и `js/auth.js`, управляющий состоянием сессии исследователя и правами доступа к экранам терминала.

2.1. Модульный сетевой клиент (`js/api.js`)

Если вызывать стандартную функцию `fetch()` напрямую из обработчиков событий кнопок или форм, код быстро обрастает дублированием. В каждом месте приходится вручную прописывать базовый URL сервера, задавать заголовок `'Content-Type': 'application/json'`, сериализовать тело запроса через `JSON.stringify()` и разбирать ответ через `.json()`.

Кроме того, у нативного `fetch()` есть особенность, о которой часто забывают: промис не переходит в состояние ошибки (`reject`) при получении HTTP-статусов ошибок вроде `404 Not Found` или `500 Internal Server Error`. Он падает в блок `catch` только при физическом обрыве соединения или блокировке запроса браузером (например, из-за CORS).

Для устранения этой рутины и централизации обработки сетевых сбоев я разработал универсальный модуль `window.api`. В его основе лежит внутренняя функция `request()`, которая:

1. Автоматически склеивает базовый адрес `http://localhost:3000` с переданным эндпоинтом.
2. Прикрепляет обязательные заголовки для работы с JSON.
3. Явно проверяет флаг `response.ok`. Если сервер вернул код за пределами диапазона 200-299, функция выбрасывает исключение с точным текстом и кодом ошибки.
4. Оборачивает сетевой вызов в блок `try...catch` и логирует ошибки в консоль браузера перед пробросом наружу.

На базе этой функции модуль экспортирует удобные методы: `api.get()`, `api.post()`, `api.patch()` и `api.delete()`.

Ниже приведен листинг универсального HTTP-клиента `js/api.js`:

```
JavaScript
const API_BASE_URL = 'http://localhost:3000';

async function request(endpoint, options = {}) {
  const url = `${API_BASE_URL}${endpoint}`;

  const headers = {
    'Content-Type': 'application/json',
    ...options.headers
  };

  const config = {
```

```

        ...options,
        headers
    };

    try {
        const response = await fetch(url, config);

        // Проверка статуса ответа сервера
        if (!response.ok) {
            throw new Error(`Ошибка HTTP: ${response.status}
${response.statusText}`);
        }

        return await response.json();
    } catch (error) {
        console.error(`Сетевой сбой при запросе к ${url}:`, error);
        throw error;
    }
}

// Публичный интерфейс клиента для работы с REST API
window.api = {
    get(endpoint) {
        return request(endpoint, { method: 'GET' });
    },
    post(endpoint, data) {
        return request(endpoint, {
            method: 'POST',
            body: JSON.stringify(data)
        });
    },
    patch(endpoint, data) {
        return request(endpoint, {
            method: 'PATCH',
            body: JSON.stringify(data)
        });
    },
    delete(endpoint) {
        return request(endpoint, { method: 'DELETE' });
    }
};

```

2.2. Механизм клиентской аутентификации и сессий (js/auth.js)

Поскольку `json-server` не содержит встроенного механизма генерации криптографических JWT-токенов, я организовал подсистему сессионного контроля на связке REST-запросов и Web Storage API (`localStorage`). Все методы сгруппированы в объекте `window.auth`.

Алгоритм регистрации нового кадета (`register.html`)

При отправке формы регистрации скрипт выполняет двухэтапную процедуру:

1. **Проверка уникальности почты:** перед созданием записи отправляется поисковый запрос `GET /users?email={email}`. Если сервер возвращает непустой массив, процесс прерывается с выводом ошибки «Исследователь с таким email уже зарегистрирован».
2. **Формирование досье:** если email свободен, генерируется объект нового кадета с уникальным кодом позывного (`NT-XXXX-X`), начальным статусом `Cadet Class`, ссылкой на персональный аватар сервиса UI-Avatars и дефолтными настройками каюты.
3. **Сохранение:** объект отправляется на сервер методом `api.post('/users', newUser)`. В ответ `json-server` присваивает пользователю уникальный числовой `id` и возвращает статус `201 Created`. Полученный объект немедленно сохраняется в `localStorage`, после чего кадет перенаправляется в кабинет.

Алгоритм входа в систему (`login.html`)

При авторизации выполняется запрос `GET /users?email={email}&password={password}`. Если пользователь с такой комбинацией почты и ключа доступа существует, `json-server` отдает массив из одного элемента. Скрипт записывает эти данные в ключ `nova_user` локального хранилища через `setCurrentUser()` и открывает доступ к терминалу. При неверных данных выводится предупреждение об ошибке.

Барьер авторизации

Для защиты закрытых разделов сервиса от несанкционированного доступа я разработал метод `requireAuth()`. Он вызывается в самом начале инициализации скрипта кабинета (`dashboard.js`). Если ключ `nova_user` в `localStorage` отсутствует или поврежден, функция мгновенно прерывает выполнение последующего кода и перенаправляет браузер на экран входа `login.html`.

Ниже представлен листинг реализации метода проверки и удержания сессии:

```
JavaScript
const AUTH_STORAGE_KEY = 'nova_user';

window.auth = {
  getCurrentUser() {
    const raw = localStorage.getItem(AUTH_STORAGE_KEY);
    try {
      return raw ? JSON.parse(raw) : null;
    } catch (e) {
```

```

        return null;
    }
},

setCurrentUser(user) {
    localStorage.setItem(AUTH_STORAGE_KEY, JSON.stringify(user));
},

logout() {
    localStorage.removeItem(AUTH_STORAGE_KEY);
    window.location.href = 'login.html';
},

// Барьер доступа
requireAuth() {
    const user = this.getCurrentUser();
    if (!user) {
        window.location.href = 'login.html';
        return null;
    }
    return user;
}
};

```

Динамическая адаптация навигационной панели (**initNavbar**)

Чтобы элементы управления соответствовали текущему статусу пользователя, метод **initNavbar()** анализирует состояние хранилища при загрузке каждой страницы:

- Если пользователь **не авторизован**: ссылки на личный кабинет (**dashboard.html**) в верхнем меню и кнопка профиля скрываются утилитарным классом Bootstrap **d-none**. Кнопки «Войти» и «Регистрация» остаются видимыми.
- Если исследователь **авторизован**: кнопки входа и регистрации скрываются, в меню открывается доступ к кабинету, а в правом углу навбара отрисовывается миниатюра аватара и реальный позывной кадета.

При нажатии на кнопку «Завершить сессию» вызывается метод **auth.logout()**, который очищает ключ в **localStorage** и возвращает пользователя на терминал авторизации.

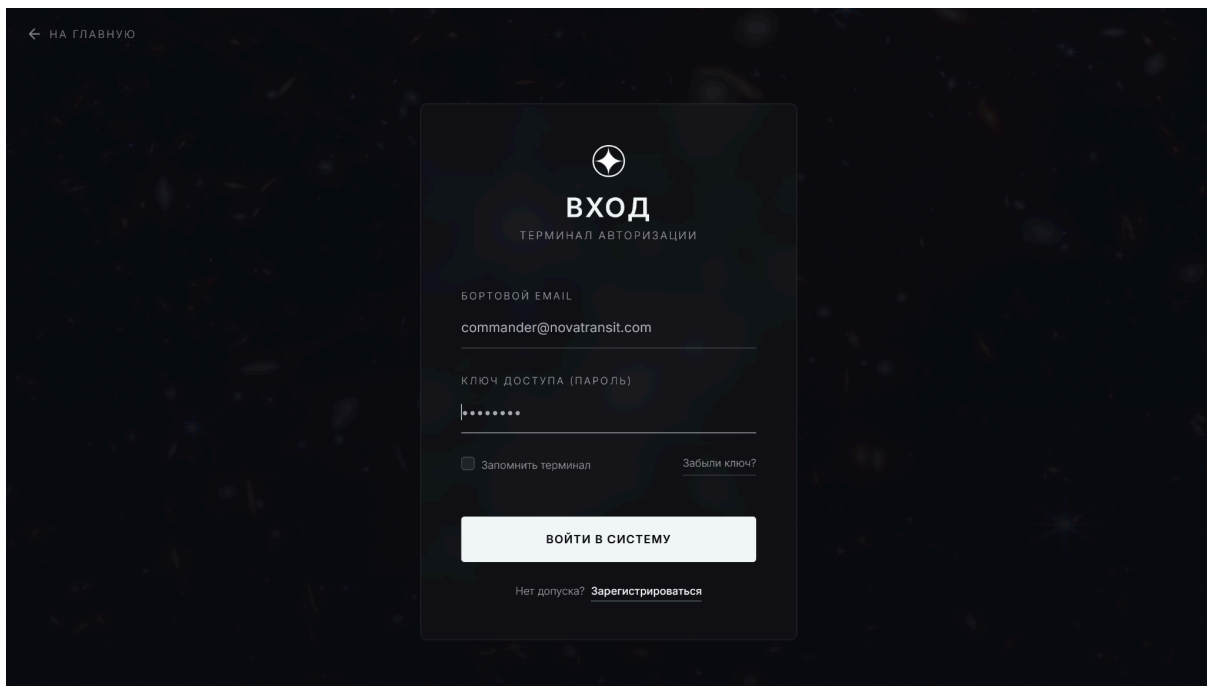


Рисунок 2 - Экран авторизации *login.html*

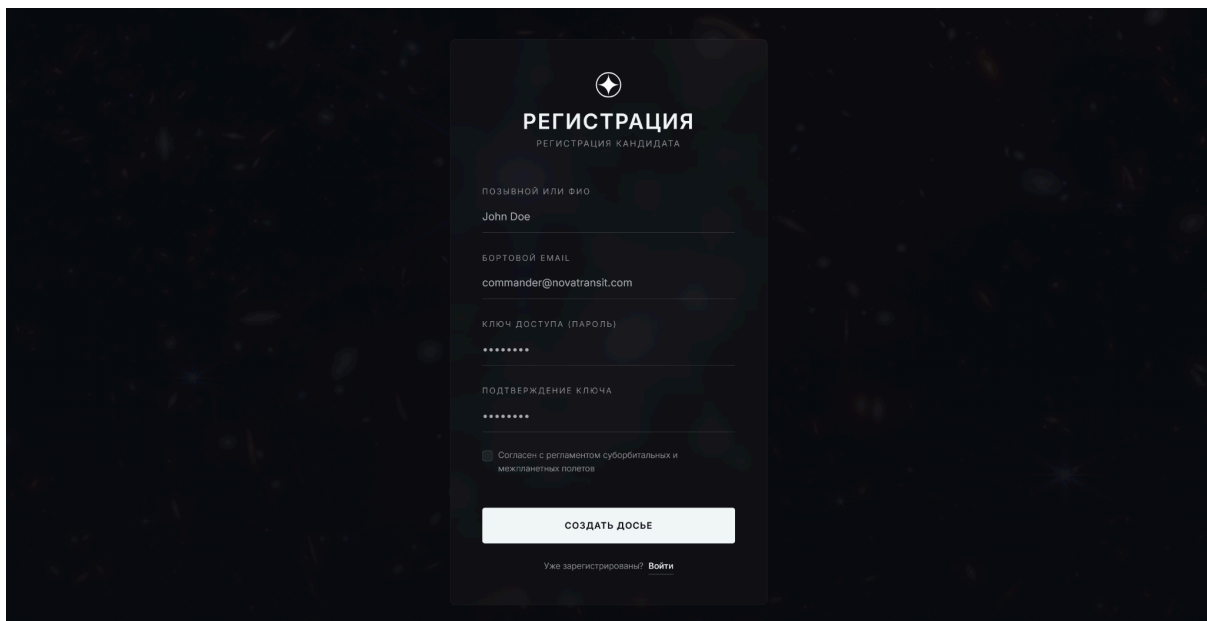


Рисунок 3 - Регистрация кадета в *register.html*

3. Динамический интерфейс и сетевое взаимодействие

3.1. Динамический каталог экспедиций (js/catalog.js)

В первой лабораторной работе все шесть карточек направлений были жестко прописаны в разметке страницы `index.html`. Это создавало массу неудобств: при изменении стоимости тура или добавлении новой планеты приходилось бы вручную редактировать громоздкий HTML-код.

В рамках ЛР2 я полностью очистил контейнер `#missionsGrid` от статических блоков, оставив его пустой точкой монтирования, а всю логику получения и отображения данных вынес в скрипт `js/catalog.js`.

Асинхронное получение данных и обработка состояний

При загрузке главной страницы вызывается асинхронная функция `loadMissions()`:

1. Контейнеру каталога временно присваивается разметка с центрированным индикатором загрузки и текстом «Установка связи с базой данных...». Это улучшает UX и дает пользователю понять, что приложение активно обращается к сети.
2. С помощью вызова `await window.api.get('/tours')` клиент отправляет запрос к мок-серверу и получает актуальный массив экспедиций из коллекции `tours`.
3. Если сервер по какой-то причине недоступен (например, забыли выполнить `npm run server`), блок `catch` перехватывает сбой и выводит аккуратное информационное сообщение об ошибке подключения к бортовой сети.

Процедурный рендеринг и сохранение селекторов фильтрации

После успешного ответа сервера массив туров преобразуется в HTML-строку через метод `.map(renderTourCard).join("")` и вставляется в DOM-дерево.

Чтобы не сломать механизм фильтрации, разработанный в первой работе, я сохранил в генерируемой верстке те же самые классы Bootstrap (`glass-panel`, `badge-gold`, `col mission-card-item`) и пользовательские data-атрибуты:

- `data-destination` - пункт назначения (earth, moon, mars, enceladus);
- `data-budget` - класс миссии (standard, premium, vip);
- `data-duration` - длительность полета (short, long).

Благодаря этому алгоритм функции `filterMissions()`, считывающий значения выпадающих списков и переключающий класс `.d-none`, заработал с динамически созданными карточками без каких-либо изменений в логике сравнения.

Ниже представлен листинг функции шаблонизации карточки экспедиции `renderTourCard()`:

JavaScript

```
function renderTourCard(tour) {
  return `
    <div class="col mission-card-item"
      data-destination="${tour.destination}"
      data-budget="${tour.budget}"
      data-duration="${tour.duration}">
      <div class="glass-panel h-100 p-4 d-flex flex-column
justify-content-between">
        <div>
          <div class="d-flex justify-content-between
align-items-center mb-3">
            <span class="${tour.badgeClass} px-2 py-1 fs-7
rounded">${tour.badge}</span>
            <span class="text-muted-custom
fs-7">${tour.code}</span>
          </div>
          <h4 class="text-main fs-5 mb-2">${tour.title}</h4>
          <p class="text-muted-custom fs-7
mb-4">${tour.shortDesc}</p>

          <div class="border-top border-secondary
border-opacity-25 pt-3 mb-4">
            <div class="d-flex justify-content-between fs-7
mb-2">
              <span
class="text-muted-custom">Дистанция:</span>
              <span
class="text-main">${tour.specs.distance}</span>
            </div>
            <div class="d-flex justify-content-between fs-7
mb-2">
              <span
class="text-muted-custom">Гравитация:</span>
              <span
class="text-main">${tour.specs.gravity}</span>
            </div>
            <div class="d-flex justify-content-between
fs-7">
              <span
class="text-muted-custom">Длительность:</span>
              <span
class="text-main">${tour.durationText}</span>
            </div>
          </div>
        </div>
      </div>
    `
}
```

```

        <div>
            <div class="d-flex justify-content-between
align-items-center mb-3">
                <span class="text-muted-custom
fs-7">Стоимость</span>
                <span class="fs-5 fw-bold
text-main">${tour.price.toLocaleString('ru-RU')} $</span>
            </div>
            <a href="tour.html?id=${tour.id}" class="btn
btn-glass w-100 py-2 fs-7">Подробнее</a>
        </div>
    </div>
</div>
`
;
}

```

Сохранение интерактивных возможностей первого экрана

Перевод каталога на API никак не затронул визуальные эффекты в Hero-секции. В файле `catalog.js` я сохранил плавную интерполяцию планет в блоке `.planet-wrapper`: при смене значения в селекте направлений активному изображению назначается класс `.active`, активирующий CSS-переход прозрачности, масштабирования и размытия. Также сохранилось динамическое свечение поискового терминала (`glow-premium` и `glow-vip`) при выборе соответствующих тарифов.

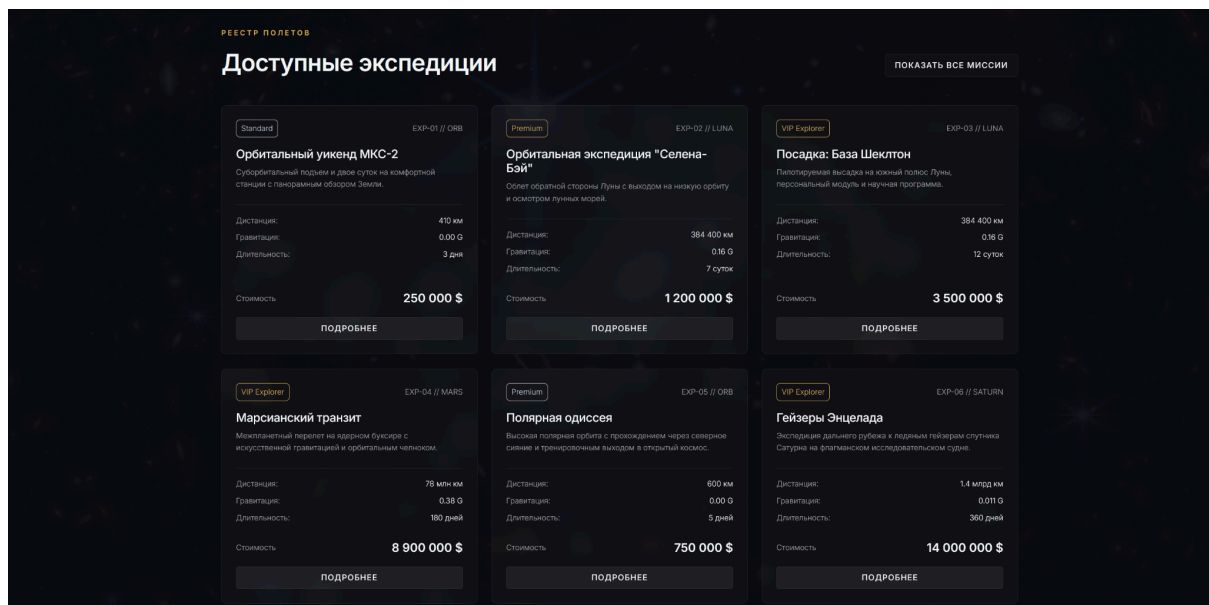




Рисунок 4 - Главная страница `index.html`: каталог экспедиций, динамически построенный из ответа `GET /tours`

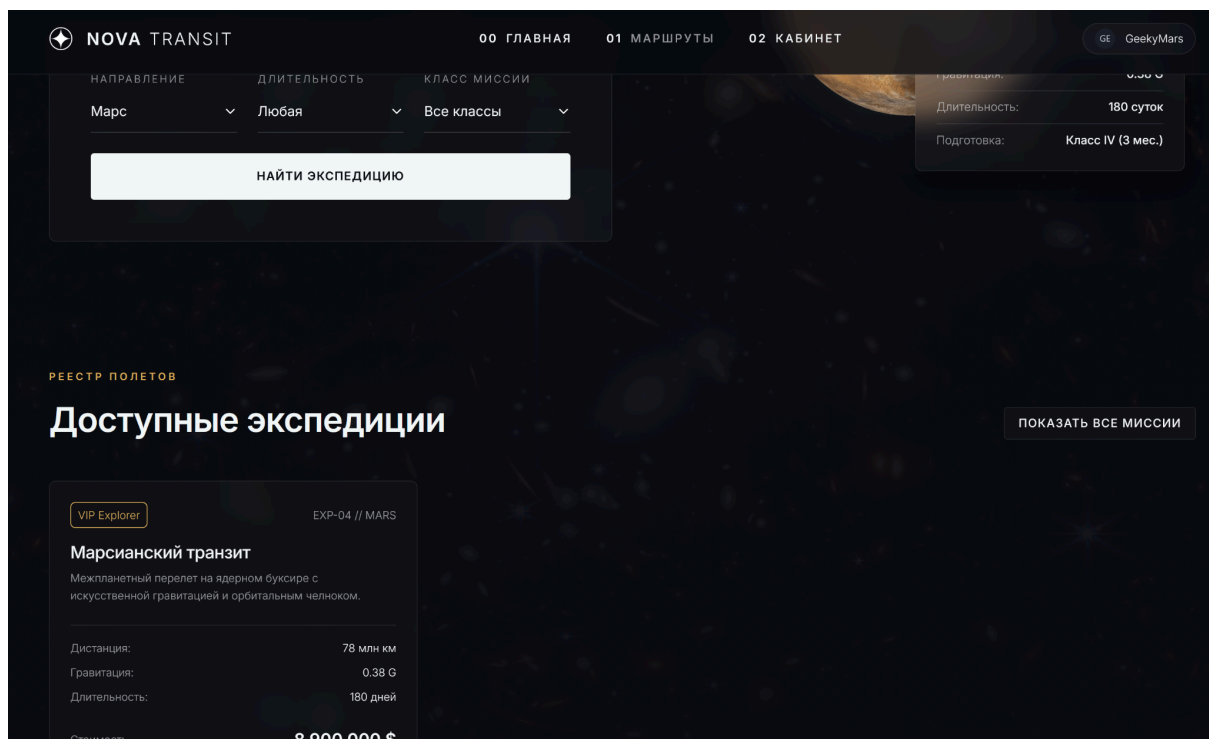


Рисунок 5 - Работа многопараметрической фильтрации каталога по полученному массиву данных

3.2. Карточка миссии и оформление бронирования (`js/tour.js`)

В первой лабораторной работе для демонстрации работы страницы `tour.html` использовался статичный JS-объект `toursData`, зашитый прямо в тело документа. При переходе на клиент-серверную архитектуру я полностью удалил этот хардкод, переведя страницу экспедиции на взаимодействие с эндпоинтами `/tours/:id` и `/bookings`.

Параметризация маршрута и обработка ошибки 404 Not Found

При открытии страницы скрипт `js/tour.js` инициализирует сценарий загрузки:

1. Через интерфейс `URLSearchParams(window.location.search)` из адресной строки считывается значение GET-параметра `id` (например, `tour.html?id=mars`). Если параметр не передан вовсе, по умолчанию выбирается базовый идентификатор `moon`.
2. Клиент отправляет асинхронный запрос `await window.api.get('/tours/' + tourId)`.
3. Если пользователь вручную ввел в URL несуществующий идентификатор (например, `?id=pluto`), сервер возвращает ошибку `404 Not Found`. В блоке `catch` я предусмотрел сценарий восстановления: скрипт перехватывает исключение, выводит предупреждение в консоль и автоматически делает резервный запрос за лунной программой по умолчанию (`/tours/moon`). Это исключает падение страницы с белым экраном и гарантирует стабильный пользовательский опыт.

Процедурное наполнение медиа-слайдера, телеметрии и графика полета

Полученный с сервера JSON-объект распределяется по интерфейсным узлам:

- **Базовые параметры и телеметрия:** текстовые поля, шифр полета, класс миссии и блок из четырех критических характеристик безопасности (дистанция, сила тяжести, перегрузка и радиационный фон в зивертах) динамически заполняются через свойства `textContent`.
- **Фотохроника:** скрипт выполняет обход массива `currentTour.slides` и обновляет ссылки на изображения `src`, а также заголовки и описания в блоках `.carousel-caption`.
- **График экспедиции:** контейнер `#timelineContainer` очищается, после чего массив контрольных точек `currentTour.timeline` преобразуется в цепочку узлов с иконками `bi-circle-fill`, временными засечками (от предполетного карантина Т-14 дней до финального входа в атмосферу) и разделительными направляющими линиями.

Модальное окно бронирования и оформление контракта

Диалоговое окно `#bookingModal` вызывается по клику на кнопку «Забронировать место». Я связал форму с профилем текущего исследователя и настроил отправку данных в базу:

- **Автоподстановка данных кадета:** скрипт запрашивает сессию через `window.auth.getCurrentUser()`. Если кадет уже авторизован в системе, его позывной и бортовой email автоматически подставляются в соответствующие инпуты формы, избавляя от повторного ручного ввода.
- **Калькулятор дополнительных услуг:** выпадающий список дат старта динамически наполняется доступными окнами запуска из массива `currentTour.dates`. Чекбоксы дополнительных модулей (индивидуальный выход в открытый космос EVA и система внешней съемки 8K) привязаны к событию `change`: функция `calculateTotal()` суммирует базовый тариф экспедиции со стоимостью выбранных опций и форматирует итоговое значение через `toLocaleString('ru-RU')`.

- **Отправка контракта на сервер (POST /bookings):** при подтверждении формы скрипт проверяет валидность полей через `checkValidity()`. В случае успеха блокируется кнопка отправки для защиты от повторных кликов, генерируется серийный номер электронного сертификата вида `NT-XXXX-2026-EXP`, и на сервер отправляется сформированная запись методом `api.post('/bookings', newBooking)`.

После получения статуса `201 Created` модальное окно закрывается, а авторизованный пользователь перенаправляется в личный кабинет `dashboard.html`, где оформленный рейс мгновенно становится его ближайшей активной экспедицией.

Ниже представлен листинг асинхронного обработчика отправки формы бронирования:

```
JavaScript
// lab_02/js/tour.js (фрагмент)
bookingForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  e.stopPropagation();

  // Проверка ограничений HTML5 Constraint Validation
  if (!bookingForm.checkValidity()) {
    bookingForm.classList.add('was-validated');
    return;
  }

  const submitBtn =
    bookingForm.querySelector('button[type="submit"]');
  submitBtn.disabled = true;
  submitBtn.textContent = 'Оформление...';

  const finalPrice = calculateTotal();
  const selectedDate = dateSelect.value;
  const serialNumber = `NT-${Math.floor(1000 + Math.random() *
    9000)}-2026-${currentTour.code.replace(/^[a-zA-Z0-9]/g, '')}`;

  // Формирование структуры нового полетного манифеста
  const newBooking = {
    userId: currentUser ? currentUser.id : null,
    tourId: currentTour.id,
    tourCode: currentTour.code.split(' // ')[0] || currentTour.code,
    route: `Земля → ${currentTour.title.split(' ')[0]}`,
    origin: "EAR",
    originName: "Земля (KK)",
    destination: currentTour.destination.toUpperCase(),
    destinationName: currentTour.title,
    date: selectedDate,
    countdown: "30 дн. 10:00",
    ship: currentTour.ship,
```

```

    gate: "Gateway A-12",
    cabin: "Каюта 1-й категории",
    status: "Ожидает старта",
    statusBadge: "badge-gold",
    totalPrice: finalPrice,
    serialNo: serialNumber,
    active: true
  };

  try {
    await window.api.post('/bookings', newBooking);

    const modalEl = document.getElementById('bookingModal');
    bootstrap.Modal.getInstance(modalEl).hide();

    alert(`Место успешно забронировано! Номер сертификата:
    ${serialNumber}`);

    if (currentUser) {
      window.location.href = 'dashboard.html';
    } else {
      bookingForm.reset();
      bookingForm.classList.remove('was-validated');
      updatePrice();
    }
  } catch (err) {
    alert(`Ошибка оформления бронирования: ${err.message}`);
  } finally {
    submitBtn.disabled = false;
    submitBtn.textContent = 'Подтвердить бронь';
  }
});

```

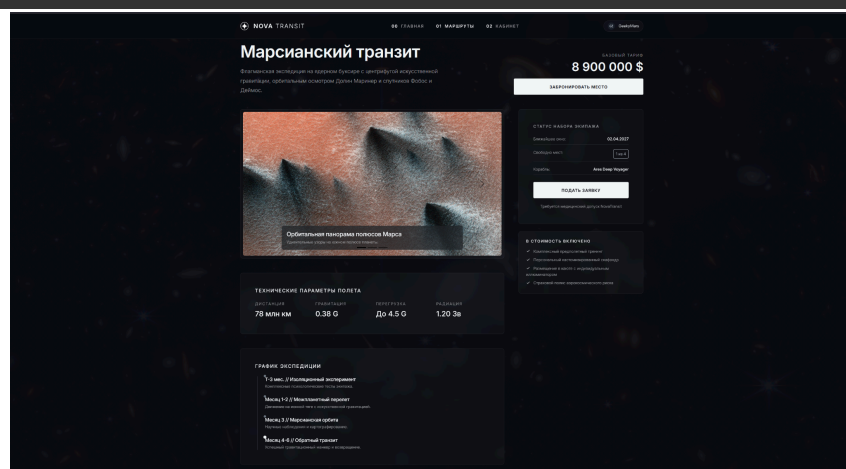


Рисунок 6 - Динамическая страница тура `tour.html?id=mars` с параметрами и графиком миссии

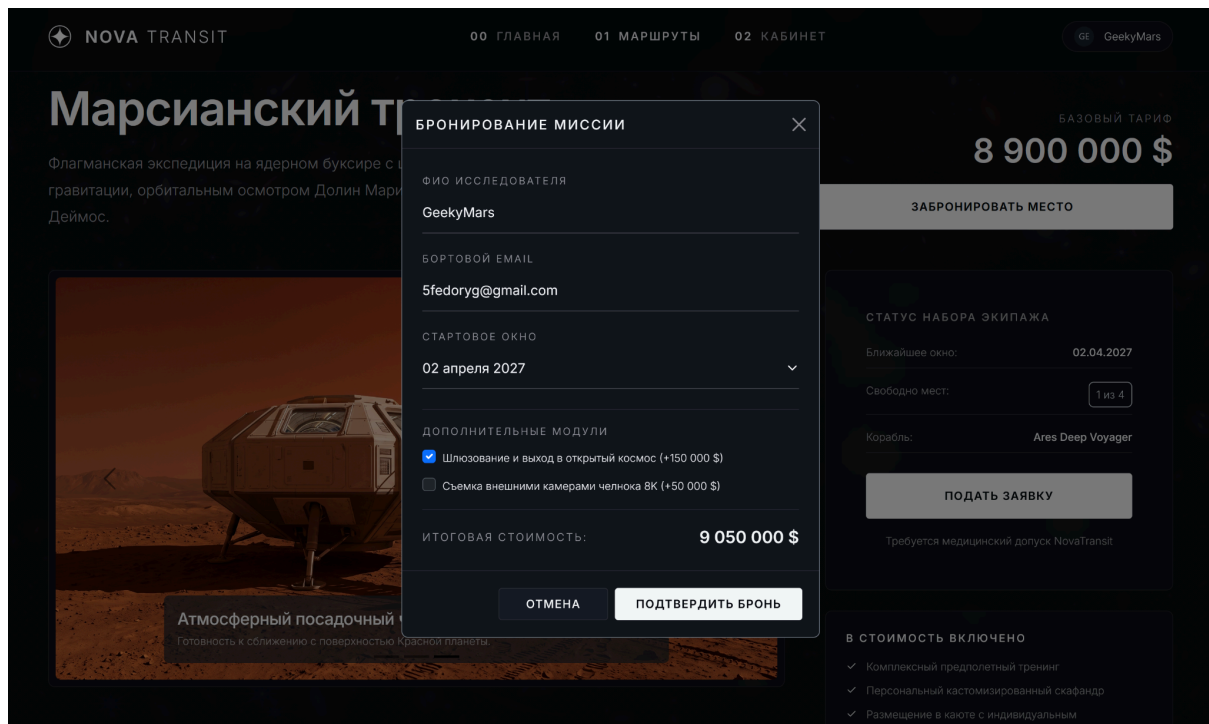


Рисунок 7 - Модальное окно бронирования с динамическим расчетом стоимости и подтверждением создания записи

3.3. Личный кабинет исследователя (js/dashboard.js)

Личный кабинет исследователя выступает центральным рабочим терминалом кадета, где концентрируются персональные данные, история полетов и управление каютой. Если в первой лабораторной работе дашборд отображал статичные данные вымышленного исследователя John Doe, то в модуле `js/dashboard.js` я полностью перевел экран на асинхронное взаимодействие с коллекциями `users` и `bookings`.

Инициализация профиля и актуализация данных (GET /users/:id)

При входе на страницу первым делом срабатывает барьер `window.auth.requireAuth()`. Если кадет не авторизован, выполнение скрипта немедленно прерывается, и пользователя перенаправляет на экран входа.

Для авторизованного исследователя запускается процедура инициализации:

1. Базовые данные берутся из локальной сессии, но для исключения рассинхронизации скрипт сразу отправляет запрос `await window.api.get('/users/' + currentUser.id)`. Это гарантирует, что интерфейс получит самые свежие параметры, даже если в базу вносились изменения из других вкладок или сессий.
2. Имя кадета, личный код допуска (`NT-XXXX-X`) и аватар подставляются в карточку «Досье исследователя».

3. Скрипт проверяет статус `user.isDeepSpaceMember`: если флаг равен `true`, с соответствующего золотого бейджа снимается класс `d-none`, подтверждая участие кадета в элитной исследовательской программе.

Полетные манифесты и бортовой журнал (GET /bookings?userId=...)

Связка пользователя с его рейсами реализована через выборку `await window.api.get('/bookings?userId=' + user.id)`:

- **Обработка отсутствия полетов:** если массив пуст (новый пользователь, только что создавший досье), скрипт скрывает посадочный талон `#activeFlightCard` и показывает блок заглушки `#noFlightsMessage` с предложением перейти в реестр экспедиций. В таблице бортового журнала при этом отображается строка о том, что записи полетов пока отсутствуют.
- **Посадочный талон активного рейса:** если у кадета есть бронирования, последний добавленный рейс (`bookings[bookings.length - 1]`) выводится в главный посадочный купон. В нем отрисовываются трехбуквенные коды космодромов (например, `EAR` → `ENC`), оставшееся время до старта, номер назначенного шлюза вылета и категория каюты.
- **Рендер таблицы бортового журнала:** все записи из полетного манифеста методом `.map()` преобразуются в строки таблицы `#bookingsTableBody` с указанием маршрута, даты, корабля-носителя и статусного бейджа («Завершен», «Ожидает старта»).

Цифровой посадочный сертификат (#ticketModal)

При нажатии на кнопку «Билет и штрихкод» в посадочном купоне открывается модальное окно `#ticketModal`. Скрипт динамически передает в сертификат код рейса, маршрут экспедиции, фамилию и позывной кадета, категорию каюты и уникальный серийный номер билета (`serialNo`), сформированный при бронировании.

Частичное обновление систем каюты (PATCH /users/:id) и уведомления Toast

Особое внимание я уделил блоку переключателей жизнеобеспечения каюты (гравитация, гипоаллергенный рацион, автозатемнение иллюминаторов).

Здесь применен метод **PATCH**. Он передает на сервер только изменившийся фрагмент объекта:

```
JSON
{
  "cabinSettings": {
    "gravity": true,
    "hypoallergenicFood": true,
    "windowDimming": true
  }
}
```

Чтобы интерфейс откликнулся мгновенно, я применил подход Optimistic UI:

1. При щелчке по тумблеру скрипт не дожидается ответа от сервера, а сразу вызывает всплывающее сообщение `telemetryToast` через `bootstrap.Toast.getOrCreateInstance()`. В заголовке тоста выводится сообщение от бортового компьютера с новым статусом («АКТИВИРОВАН» или «ДЕАКТИВИРОВАН»).
2. Параллельно в фоне отправляется асинхронный сетевой запрос `window.api.patch('/users/' + user.id, ...)`.
3. Полученный обновленный объект сохраняется в локальную сессию. Если же произошел сбой сети, блок `catch` перехватывает ошибку, возвращает тумблер в исходное состояние и информирует пользователя о неудаче.
4. Благодаря правилу игнорирования `db.json` в настройках Live Server перезапись базы данных сервером происходит полностью бесшовно, без дергания и перезагрузки страницы.

Ниже представлен листинг обработчика переключения тумблеров и отправки PATCH-запроса:

```
JavaScript
// Функция гарантированного вызова уведомления
function showTelemetryToast(message) {
  const toastEl = document.getElementById('telemetryToast');
  const msgEl = document.getElementById('toastMessage');
  if (!toastEl) return;

  if (msgEl) msgEl.textContent = message;

  const toastInstance = bootstrap.Toast.getOrCreateInstance(toastEl, {
    delay: 3000,
    autohide: true
  });
  toastInstance.show();
}

// Привязка тумблеров систем жизнеобеспечения
const switches = {
  gravSwitch: { key: 'gravity', name: 'Режим гравитации' },
  foodSwitch: { key: 'hypoallergenicFood', name: 'Бортовой рацион' },
  dimSwitch: { key: 'windowDimming', name: 'Затемнение иллюминатора' }
};

document.querySelectorAll('.telemetry-switch').forEach(toggle => {
  toggle.addEventListener('change', async (e) => {
    const meta = switches[e.target.id];
    if (!meta) return;
```

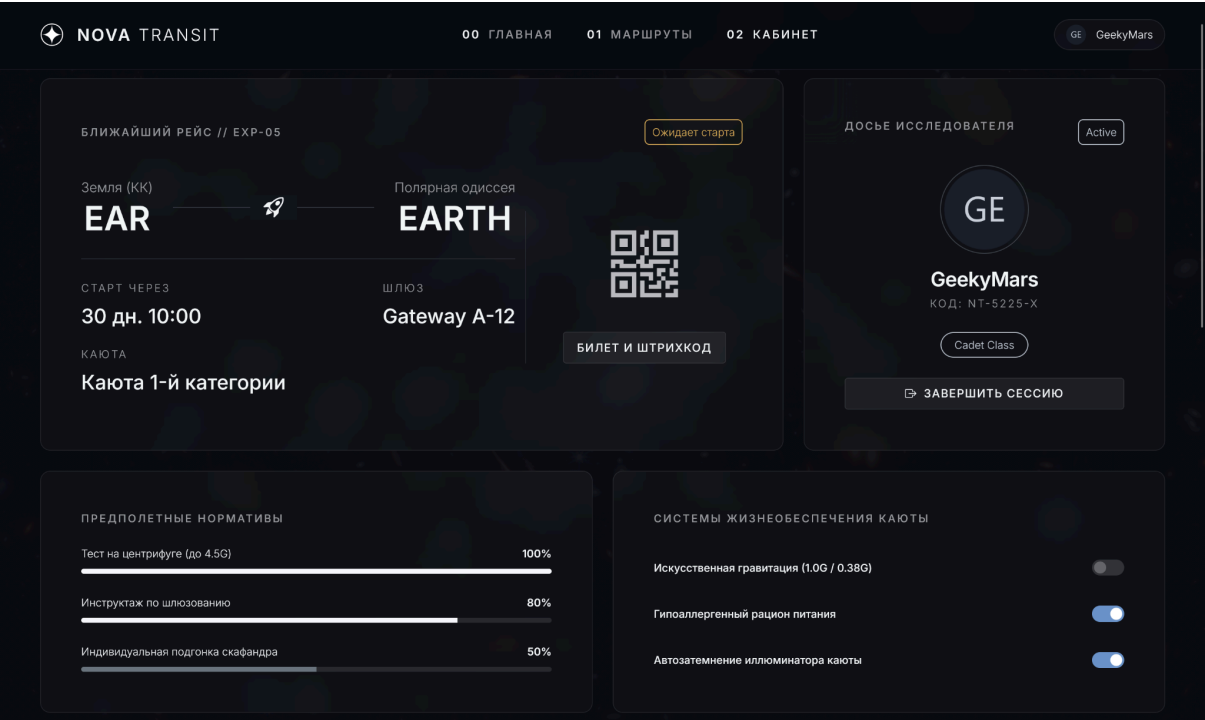
```
    const isChecked = e.target.checked;
    const stateText = isChecked ? 'АКТИВИРОВАН' :
'ДЕАКТИВИРОВАН';

    // 1. Мгновенная визуальная обратная связь
    showTelemetryToast(`${meta.name}: статус изменен на
[${stateText}].`);

    // 2. Фоновая отправка PATCH-запроса на сервер
    try {
        const currentSettings = user.cabinSettings || {};
        const updatedSettings = {
            ...currentSettings,
            [meta.key]: isChecked
        };

        const updatedUser = await
window.api.patch(`/users/${user.id}`, {
            cabinSettings: updatedSettings
        });

        user = updatedUser;
        window.auth.setCurrentUser(updatedUser);
    } catch (err) {
        console.error('Ошибка синхронизации телеметрии:', err);
        e.target.checked = !isChecked; // Откат тумблера при сбое
        showTelemetryToast('Сбой связи: параметры каюты не
сохранены.');
```



БОРТОВОЙ ЖУРНАЛ ЭКСПЕДИЦИЙ

НАПРАВЛЕНИЕ	ДАТА	КОРАБЛЬ	СТАТУС
🚀 Земля → Гейзеры	10 августа 2028	Titan Horizon Titan-Class	Ожидает старта
🚀 Земля → Полярная	12 января 2027	Aurora Polar Cruiser	Ожидает старта

Рисунок 8 - Интерфейс личного кабинета dashboard.html с активным билетом кадета и журналом полетов

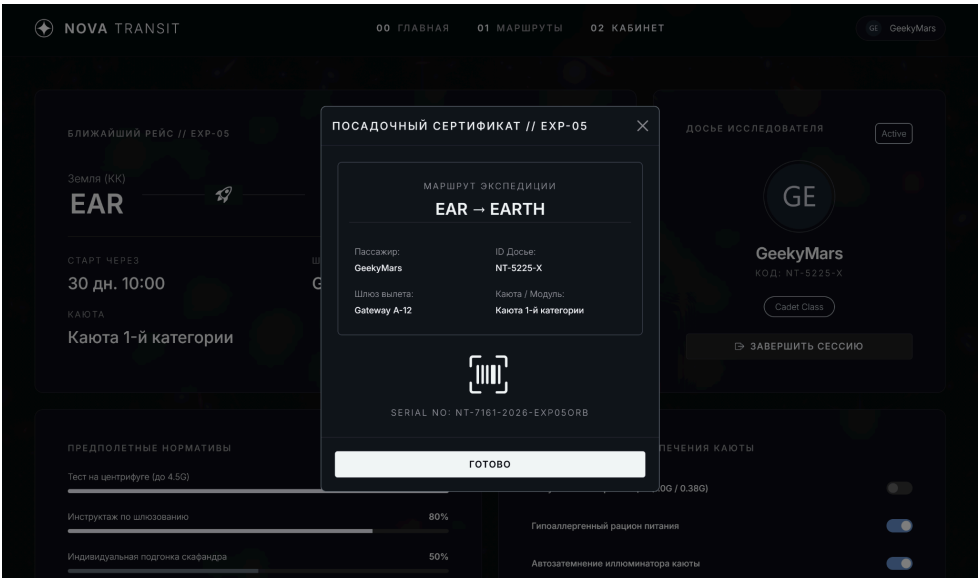


Рисунок 9 - Модальное окно цифрового посадочного сертификата с персональным досье

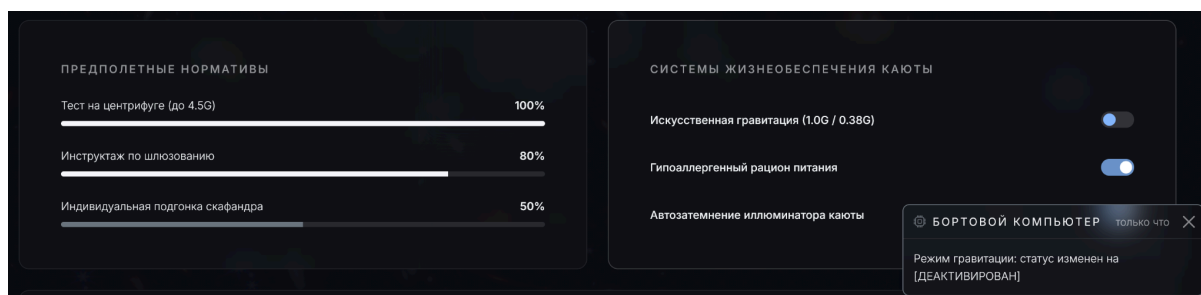


Рисунок 10 - Момент переключения тумблера гравитации: появление всплывающего сообщения бортового компьютера без перезагрузки страницы

4. Интеграция внешнего публичного сервиса (NASA APOD API)

4.1. Архитектура интеграции со сторонней платформой (js/nasa.js)

Чтобы проект не ограничивался только локальным мок-сервером, а решал задачу взаимодействия с реальной сетевой инфраструктурой, я интегрировал сторонний публичный REST API.

В качестве внешнего сервиса я выбрал официальный программный интерфейс NASA - **APOD (Astronomy Picture of the Day)**. Выбор продиктован концепцией NovaTransit: сервис бронирования межпланетных полетов органично дополняется ежедневной сводкой глубокого космоса от реального космического агентства. Виджет размещен в нижней части личного кабинета исследователя ([dashboard.html](#)) под заголовком «Внешний телеметрический шлюз // NASA APOD».

Спецификация внешнего эндпоинта:

- URL: [\[https://api.nasa.gov/planetary/apod\]](https://api.nasa.gov/planetary/apod)(<https://api.nasa.gov/planetary/apod>)
- Метод: GET
- Ключевые параметры запроса:
 - **api_key** - ключ доступа к сервису (строка). По умолчанию NASA предоставляет публичный ключ **DEMO_KEY** для быстрого прототипирования.
- Структура получаемого ответа (JSON):
 - **title** (строка) - официальное название астрономического объекта или явления;
 - **date** (строка формата **YYYY-MM-DD**) - дата публикации снимка;
 - **explanation** (строка) - экспертный комментарий профессиональных астрофизиков с описанием запечатленных процессов;

- **media_type** (строка) - тип контента (**image** для фотоснимков либо **video** для видеороликов и симуляций);
- **url** и **hdurl** (строки) - прямые ссылки на медиафайл в стандартном и ультравысоком разрешении.

4.2. Преодоление ограничений сети и политики CORS

В процессе подключения внешнего API возникла сложная техническая проблема, связанная с политикой ограничения одного источника (Same-Origin Policy / CORS) и квотами публичного шлюза NASA.

Анализ проблемы исчерпания лимитов и CORS-блокировки

Публичный ключ **DEMO_KEY** жестко ограничен квотой: не более 30-50 обращений в час. Скорее всего, поскольку этот ключ используется разработчиками и студентами по всему миру, лимит почти всегда оказывается исчерпанным.

Реализация каскадного шлюза

Для обеспечения гарантированной отказоустойчивости приложения я спроектировал трехэтапную цепочку получения данных (Fallback Pipeline):

1. **Первичный запрос:** клиент отправляет стандартный вызов к прямому эндпоинту **api.nasa.gov** с ключом **DEMO_KEY** (или персональным API-ключом).
2. **Переключение на CORS-шлюз:** если прямой запрос завершается со статусом ошибки (например, **403 Forbidden** или **429 Too Many Requests**), скрипт мгновенно перехватывает управление и отправляет резервный запрос через открытый CORS-прокси (**[https://corsproxy.io/](https://corsproxy.io/?...)**). Прокси-сервер обращается к NASA от своего имени и транслирует ответ в браузер, гарантированно подставляя корректный заголовок **Access-Control-Allow-Origin: ***.
3. **Автономный архивный режим:** если у пользователя полностью отсутствует подключение к интернету или внешние шлюзы заблокированы, блок **catch** предотвращает падение скрипта и бесшовно переключает виджет на локальный кэш. На экран выводится сохраненная орбитальная панорама Земли с бейджем **Offline Cache** и пояснительным текстом.

Адаптация медиа-контейнера под разнородный контент

В ходе тестирования интерфейса я внес два важных эргономических улучшения:

- **Поддержка видео:** сервис NASA иногда публикует вместо статичных фото видеоролики с YouTube или Vimeo. Скрипт анализирует поле **media_type**: если значение равно **video**, в DOM монтируется адаптивный фрейм **<iframe>**, а для фотографий

используется тег `<img>`.

- **Увеличение высоты области просмотра:** изначально высота медиа-блока составляла 280px, из-за чего широкоформатные панорамы и вертикальные снимки туманностей обрезались по краям. Я увеличил высоту контейнера в 1,5 раза - до **420px**, применив CSS-свойство `object-fit: cover` и контрастную подложку `bg-black bg-opacity-25`. Это позволило отображать снимки глубокого космоса целиком, сохранив строгие пропорции и четкую читаемость описания под ними.

Ниже приведен листинг модуля `js/nasa.js` с каскадной обработкой ошибок:

```
JavaScript
document.addEventListener('DOMContentLoaded', async () => {
  const apodContainer = document.getElementById('nasaApodContent');
  const apodSpinner = document.getElementById('nasaApodSpinner');
  if (!apodContainer) return;

  const API_KEY = 'DEMO_KEY';
  const PRIMARY_URL =
`https://api.nasa.gov/planetary/apod?api_key=${API_KEY}`;
  const FALLBACK_URL = 'https://corsproxy.io/?' +
encodeURIComponent(`https://api.nasa.gov/planetary/apod?api_key=DEMO_KEY`);

  let data = null;

  try {
    let response = await fetch(PRIMARY_URL);

    // Каскадное переключение при исчерпании лимитов DEMO_KEY
    if (!response.ok) {
      console.warn(`NASA API вернул статус ${response.status}.
Переключение на резервный канал...`);
      response = await fetch(FALLBACK_URL);
    }

    if (!response.ok) {
      throw new Error(`Сбой внешнего шлюза: ${response.status}`);
    }

    data = await response.json();
  } catch (error) {
    console.warn('Внешний шлюз недоступен, активирован автономный
режим:', error);
  } finally {
    if (apodSpinner) apodSpinner.classList.add('d-none');
  }
}
```

```

}

// Рендеринг полученных данных
if (data && (data.url || data.hdur1)) {
    const mediaUrl = data.hdur1 || data.url

    const mediaHtml = data.media_type === 'video'
        ? `
&amp;amp;amp;lt;/div&amp;amp;amp;gt;
```

```

    </div>
    <p class="text-muted-custom fs-7 mb-0">
        Внешний шлюз временно недоступен. Отображаются
        архивные данные орбитального телескопа NovaTransit.
    </p>
    `;
  }
});

```

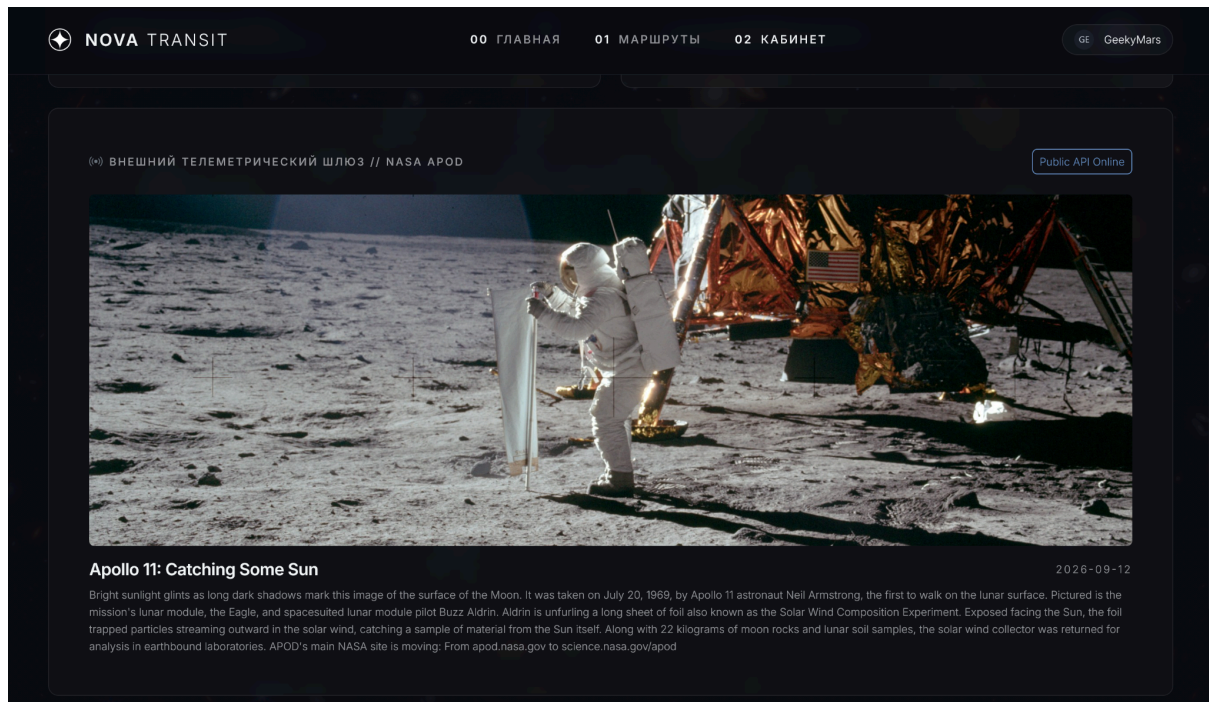


Рисунок 11 - Виджет внешнего шлюза NASA APOD в личном кабинете с актуальным снимком дня и астрофизическим описанием

Заключение

В ходе выполнения лабораторной работы № 2 я завершил перевод статического веб-сервиса NovaTransit на полноценную динамическую клиент-серверную архитектуру. Проект перестал быть просто набором сверстаных экранов и превратился в интерактивное веб-приложение, работающее с базой данных и внешними сетевыми сервисами.

Основные результаты работы:

- **Клиент-серверная архитектура и эмуляция базы данных:** с помощью **json-server** я развернул локальный REST API, поддерживающий стандартные операции CRUD. Спроектированная структура **db.json** обеспечила реляционные связи между досье исследователей (**users**), каталогом миссий (**tours**) и созданными контрактами (**bookings**).

- **Асинхронное взаимодействие:** весь сетевой слой переведен на асинхронный синтаксис `async/await` и нативный Fetch API. Созданный модуль `api.js` централизовал выполнение запросов (`GET`, `POST`, `PATCH`), автоматическую сериализацию JSON-структур и сквозную обработку кодов состояний HTTP.
- **Управление сессией и безопасность маршрутов:** в модуле `auth.js` я реализовал клиентскую аутентификацию на базе `localStorage` без привлечения сторонних библиотек. Механизм Route Guard гарантирует защиту закрытого терминала личного кабинета (`dashboard.html`) от неавторизованных пользователей, а навигационная панель автоматически адаптируется под текущий статус исследователя.
- **Динамический интерфейс и частичные обновления:** каталог экспедиций и детальная страница полета теперь генерируются процедурно из данных сервера. Реализовано оформление бронирования методом `POST`, а тумблеры систем жизнеобеспечения каюты синхронизируются через метод `PATCH` с мгновенным откликом интерфейса (Optimistic UI) и всплывающими сообщениями Bootstrap Toast.
- **Интеграция стороннего API и отказоустойчивость:** в личный кабинет успешно интегрирован публичный сервис NASA APOD. На практике разобравшись с механизмом блокировок CORS и исчерпанием лимитов публичных ключей, я построил каскадный шлюз (Fallback Pipeline) с автоматическим переключением на прокси и автономный архивный режим (Offline Cache).

Выполнение этой работы дало мне отличный практический опыт. Больше всего мне понравилось настраивать взаимодействие с базой данных и сразу видеть живой результат в интерфейсе: когда оформленный рейс тут же появляется в полетном манифесте кабинета, переключатели каюты сохраняют свое состояние в `db.json`, а каталог оживает на реальных данных. Видеть, как статичная верстка превращается в полноценно работающую систему с сохранением состояния между сессиями, оказалось действительно увлекательным процессом.